

fore, the orthogonal matrix R_0 in (39) is not unique either. (Though not included in this paper, numerical examples have confirmed this.)

REFERENCES

- [1] L. B. Jackson, "Roundoff-noise analysis for fixed-point digital filters realized in cascade or parallel form," *IEEE Trans. Audio Electroacoust.*, vol. AU-18, pp. 107-122, June 1970.
- [2] W. S. Lee, "Optimization of digital filters for low roundoff noise," *IEEE Trans. Circuits Syst.*, vol. CAS-21, pp. 424-431, May 1974.
- [3] S. Y. Hwang, "On optimization of cascade fixed-point digital filters," *IEEE Trans. Circuit Theory*, vol. CT-21, pp. 163-166, Jan. 1974.
- [4] —, "Realization of canonical digital networks," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-22, pp. 27-39, Feb. 1974.
- [5] B. Liu, "Effect of finite-word-length on the accuracy of digital filters—A review," *IEEE Trans. Circuit Theory*, vol. CT-18, pp. 670-677, Nov. 1971.
- [6] S. Y. Hwang, "Roundoff noise in state-space digital filtering—A general analysis," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-24, pp. 256-262, June 1976.
- [7] —, "Dynamic range constraint in state-space digital filtering," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-23, pp. 591-593, Dec. 1975.
- [8] S. Barnett and C. Storey, *Matrix Methods in Stability Theory*. New York: Barnes and Noble, 1970.
- [9] F. R. Gantmacher, *The Theory of Matrices*. New York: Chelsea, 1959.
- [10] R. Bellman, *Introduction to Matrix Analysis*. New York: McGraw-Hill, 1970.
- [11] H. Hancock, *Theory of Maxima and Minima*. New York: Dover, 1960.
- [12] V. N. Faddeeva, *Computational Methods of Linear Algebra*. New York: Dover, 1959.
- [13] P. E. Mantey, "Eigenvalue sensitivity and state-variable selection," *IEEE Trans. Automat. Contr.*, vol. AC-13, pp. 263-269, June 1968.
- [14] C. T. Mullis and R. A. Roberts, "Filter structures which minimize roundoff noise in fixed-point digital filters," in *1976 Proc. IEEE Int. Conf. Acoust., Speech, and Signal Processing*, pp. 505-508, 1976.
- [15] —, "Synthesis of minimum roundoff noise fixed-point digital filters," *IEEE Trans. Circuits Syst.*, vol. CAS-23, pp. 551-562, Sept. 1976.

A Prime Factor FFT Algorithm Using High-Speed Convolution

DEAN P. KOLBA, STUDENT MEMBER, IEEE, AND THOMAS W. PARKS, MEMBER, IEEE

Abstract—Two recently developed ideas, the conversion of a discrete Fourier transform (DFT) to convolution and the implementation of short convolutions with a minimum of multiplications, are combined to give efficient algorithms for long transforms. Three transform algorithms are compared in terms of the number of multiplications and additions. Timing for a prime factor fast Fourier transform (FFT) algorithm using high-speed convolution, which was programmed for an IBM 370 and an 8080 microprocessor, is presented.

I. INTRODUCTION

THE calculation of the discrete Fourier transform (DFT)

$$X(k) = \sum_{n=0}^{N-1} x(n) e^{-j(2\pi/N)nk}$$

is one of the central operations in digital signal processing. The development and wide spread use of the fast Fourier transform (FFT), stimulated by the paper of Cooley and Tukey [1], has had a major impact on signal processing.

Recently, several new ideas have emerged which lead to new algorithms for the DFT. One key idea described by Rader [2] in 1968 was the observation that computation of the DFT can be changed into a circular convolution by rearranging the data

when N is prime. Thus, if one has a fast way to do convolution, he now has a fast way to do the DFT.

Winograd [3] has shown the minimum number of multiples required for circular convolution. New convolution algorithms which often achieve this minimum are being developed by Agarwal and Cooley [4].

In a concise paper Winograd [5] combines the conversion of a DFT to convolution, for prime and prime power lengths, with these new convolution algorithms for short transforms. He proposes that long transforms be computed by nesting these short, high-speed transforms and presents a table comparing the number of operations required with the conventional FFT.

This paper first reviews the two central ideas; conversion of a DFT to circular convolution and convolution with the minimum number of multiples, then presents a study of various implementations of long transforms.

An alternative to the nested algorithm proposed by Winograd, a prime factor FFT algorithm using high-speed convolution for individual factors, is singled out as a promising approach and programmed for two machines; an IBM 370 and an 8080 microprocessor. Tables compare this approach with Winograd's nesting and with the conventional power of 2 FFT. While the idea of breaking up a one-dimensional transform into a multidimensional transform with prime factors is not new (see Good [6] and Thomas [7]), the combination of short, high-speed convolution algorithms with this multidimensional expansion appears to be a promising new way to implement the DFT.

Manuscript received November 1, 1976; revised March 1, 1977. This work was supported by the National Science Foundation under Grant ENG 70-01349 A03.

The authors are with the Department of Electrical Engineering, Rice University, Houston, TX 77001.

II. DOING DFT'S WITH CONVOLUTION

The discrete Fourier transform (DFT)

$$X(k) = \sum_{n=0}^{N-1} x(n) W^{nk}, \quad k = 0, \dots, N-1$$

$$W = e^{-j(2\pi/N)} \quad (1)$$

is a linear transformation of the N -dimensional data vector

$$\mathbf{x} = \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix}$$

into the vector

$$\mathbf{X} = \begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix}$$

of frequency samples.

The matrix representation

$$\begin{bmatrix} X(0) \\ X(1) \\ \vdots \\ X(N-1) \end{bmatrix} = \begin{bmatrix} 1 & 1 & \dots & 1 \\ 1 & W^1 & W^2 & \dots & W^{N-1} \\ \vdots & W^2 & W^4 & \dots & W^{2(N-1)} \\ \vdots & \vdots & \vdots & \vdots & \vdots \\ 1 & W^{N-1} & W^{2(N-1)} & \dots & W^{(N-1)(N-1)} \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ \vdots \\ x(N-1) \end{bmatrix} \quad (2)$$

shows that the complexity of the algorithm arises from the $(N-1)$ by $(N-1)$ lower right portion of the matrix. If

$$\bar{X}(k) = \sum_{n=1}^{N-1} x(n) W^{kn}, \quad k = 1, \dots, N-1 \quad (3)$$

can be computed efficiently, then we will have a fast algorithm for computing (1), the DFT, since from (3), (1) can easily be calculated:

$$X(0) = \sum_{n=0}^{N-1} x(n)$$

$$X(k) = x(0) + \bar{X}(k), \quad k = 1, 2, \dots, N-1. \quad (4)$$

To see how (3) may be converted to circular convolution, consider the matrix representation of (3), for $N=5$ with exponents taken modulo 5:

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(3) \\ \bar{X}(4) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 & W^3 & W^4 \\ W^2 & W^4 & W^1 & W^3 \\ W^3 & W^1 & W^4 & W^2 \\ W^4 & W^3 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(2) \\ x(3) \\ x(4) \end{bmatrix} \quad (5)$$

If we interchange the last two columns of (5), and then interchange the last two rows we get

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(4) \\ \bar{X}(3) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 & W^4 & W^3 \\ W^2 & W^4 & W^3 & W^1 \\ W^4 & W^3 & W^1 & W^2 \\ W^3 & W^1 & W^2 & W^4 \end{bmatrix} \begin{bmatrix} x(1) \\ x(2) \\ x(4) \\ x(3) \end{bmatrix} \quad (6)$$

We now have something like backwards circular convolution or circular correlation. By fixing $x(1)$ and reversing the remaining input vector, we obtain the conventional circular convolution:

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(4) \\ \bar{X}(3) \end{bmatrix} = \begin{bmatrix} W^1 & W^3 & W^4 & W^2 \\ W^2 & W^1 & W^3 & W^4 \\ W^4 & W^2 & W^1 & W^3 \\ W^3 & W^4 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(3) \\ x(4) \\ x(2) \end{bmatrix} \quad (7)$$

The computation of (3) (and thus, the DFT) has been changed to a circular convolution (7) by permuting the input and the output indices. If we have a fast way of doing convolutions, we now have a fast way of doing a DFT.

This idea was first presented by Rader [2]. He showed how the permutation can always be done when N is a prime

number. Since $W^N = 1$, we are dealing with the product of integers modulo N in the exponent of W in (3). To change the DFT into a circular convolution a mapping of the indices is used to change multiplication of indices modulo N to addition of indices modulo $N-1$. The set of integers $\{1, 2, \dots, N-1\}$ forms a cyclic group with the operation of multiplication modulo N [8]. We can always find at least one integer α in the group with the property that any integer in the group may be expressed as some power of α . By ordering the data according to the exponent of α we can always change (3) to circular convolution for prime N . The relation between the new index m and the original n is

$$n = \alpha^m \text{ modulo } N, \quad \begin{matrix} n = 1, 2, \dots, N-1 \\ m = 0, 1, \dots, N-2 \end{matrix} \quad (8)$$

where

$$\alpha^k \neq 1, \quad \text{for } 0 < k < N-1, \\ \alpha^{N-1} = 1. \quad (9)$$

The permutation map (8) is an isomorphism between the multiplicative group of $N-1$ integers $\{1, 2, \dots, N-1\}$ and the additive group of $N-1$ integers $\{0, 1, \dots, N-2\}$. An integer α with property (9) is called a primitive $(N-1)$ st root of unity and is said to generate the group since any element can be written as some power of α . Just as logarithms change

multiplication to addition, (8) changes multiplication into addition of indices. When (8) is used on both input and output indices, (3) becomes

$$\bar{X}(\alpha^l) = \sum_{m=0}^{N-2} x(\alpha^m) W^{\alpha^{(l+m)}}, \quad l=0, \dots, N-2. \quad (10)$$

In (10) the exponents of α are taken modulo $N-1$. This gives for $N=5$ the backward circular convolution of (6) when $\alpha=2$. To obtain conventional circular convolution we change the sign of m in (10) which corresponds to fixing $x(\alpha^0)$ and reversing the remaining input sequence.

$$\bar{X}(\alpha^l) = \sum_{m=0}^{N-2} x(\alpha^{-m}) W^{\alpha^{(l-m)}}, \quad l=0, \dots, N-2. \quad (11)$$

Again, the indices in (11) (the exponents of α) are taken modulo $N-1$. By combining (11) with (4), we can always convert the computation of a DFT of prime length into a circular convolution.

Winograd [5] and McClellan and Rader [9] have shown that the DFT may also be converted to convolution when the transform length N is a prime power, i.e., $N=p^r$ for a prime $p \neq 2$. The conversion is a bit more complicated since we must first remove all integers which contain a factor p from the set $\{1, 2, \dots, N-1\}$ to get a cyclic group with $p^{r-1}(p-1)$ elements. This cyclic group leads to a circular convolution of length $p^{r-1}(p-1)$ as before. The remaining computation consists of two DFT's of length p^{r-1} . For example with $N=9=3^2$, we delete the integers 3 and 6 to obtain the set $\{1, 2, 4, 5, 7, 8\}$ which forms a cyclic group under multiplication modulo 9 and is isomorphic to the additive group of integers $\{0, 1, 2, 3, 4, 5\}$ under addition modulo 6. The integer 2 will generate the multiplicative group since the powers of 2 modulo 9, $2^m \bmod 9, m=0, \dots, 5$, are 1, 2, 4, 8, 7, 5.

In terms of the matrix representation

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \\ X(3) \\ X(4) \\ X(5) \\ X(6) \\ X(7) \\ X(8) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & W^1 & W^2 & W^3 & W^4 & W^5 & W^6 & W^7 & W^8 \\ 1 & W^2 & W^4 & W^6 & W^8 & W^1 & W^3 & W^5 & W^7 \\ 1 & W^3 & W^6 & 1 & W^3 & W^6 & 1 & W^3 & W^6 \\ 1 & W^4 & W^8 & W^3 & W^7 & W^2 & W^6 & W^1 & W^5 \\ 1 & W^5 & W^1 & W^6 & W^2 & W^7 & W^3 & W^8 & W^4 \\ 1 & W^6 & W^3 & 1 & W^6 & W^3 & 1 & W^6 & W^3 \\ 1 & W^7 & W^5 & W^3 & W^1 & W^8 & W^6 & W^4 & W^2 \\ 1 & W^8 & W^7 & W^6 & W^5 & W^4 & W^3 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \\ x(7) \\ x(8) \end{bmatrix} \quad (12)$$

we remove rows and columns corresponding to indices 0, 3, and 6 and compute the remaining length 6 transformation

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(4) \\ \bar{X}(5) \\ \bar{X}(7) \\ \bar{X}(8) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 & W^4 & W^5 & W^7 & W^8 \\ W^2 & W^4 & W^8 & W^1 & W^5 & W^7 \\ W^4 & W^8 & W^7 & W^2 & W^1 & W^5 \\ W^5 & W^1 & W^2 & W^7 & W^8 & W^4 \\ W^7 & W^5 & W^1 & W^8 & W^4 & W^2 \\ W^8 & W^7 & W^5 & W^4 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(2) \\ x(4) \\ x(5) \\ x(7) \\ x(8) \end{bmatrix} \quad (13)$$

using the permutation

$$n = 2^m \bmod 9, \quad \begin{matrix} n = 1, 2, 4, 8, 7, 5 \\ m = 0, 1, 2, 3, 4, 5 \end{matrix}$$

to obtain the circular convolution (with input reversed as before)

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(4) \\ \bar{X}(8) \\ \bar{X}(7) \\ \bar{X}(5) \end{bmatrix} = \begin{bmatrix} W^1 & W^5 & W^7 & W^8 & W^4 & W^2 \\ W^2 & W^1 & W^5 & W^7 & W^8 & W^4 \\ W^4 & W^2 & W^1 & W^5 & W^7 & W^8 \\ W^8 & W^4 & W^2 & W^1 & W^5 & W^7 \\ W^7 & W^8 & W^4 & W^2 & W^1 & W^5 \\ W^5 & W^7 & W^8 & W^4 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(5) \\ x(7) \\ x(8) \\ x(4) \\ x(2) \end{bmatrix} \quad (14)$$

In addition to (14) we must complete the computation for the rows and columns removed from (12). For the deleted rows we have

$$\begin{bmatrix} X(0) \\ X(3) \\ X(6) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & W^3 & W^6 & 1 & W^3 & W^6 & 1 & W^3 & W^6 \\ 1 & W^6 & W^3 & 1 & W^6 & W^3 & 1 & W^6 & W^3 \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \\ x(3) \\ x(4) \\ x(5) \\ x(6) \\ x(7) \\ x(8) \end{bmatrix} \quad (15)$$

Since

$$W^3 = e^{-j(2\pi/9) \cdot 3} = e^{-j(2\pi/3)} = W_3,$$

(15) is simply a length 3 DFT of added data

$$\begin{bmatrix} X(0) \\ X(3) \\ X(6) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & W_3 & W_3^2 \\ 1 & W_3^2 & W_3 \end{bmatrix} \begin{bmatrix} x(0) + x(3) + x(6) \\ x(1) + x(4) + x(7) \\ x(2) + x(5) + x(8) \end{bmatrix}.$$

For the deleted columns we have

$$\begin{bmatrix} Y(0) \\ Y(1) \\ Y(2) \\ Y(3) \\ Y(4) \\ Y(5) \\ Y(6) \\ Y(7) \\ Y(8) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & W^3 & W^6 \\ 1 & W^6 & W^3 \\ 1 & 1 & 1 \\ 1 & W^3 & W^6 \\ 1 & W^6 & W^3 \\ 1 & 1 & 1 \\ 1 & W^3 & W^6 \\ 1 & W^6 & W^3 \end{bmatrix} \begin{bmatrix} x(0) \\ x(3) \\ x(6) \end{bmatrix} \quad (16)$$

For (16) a second length 3 DFT can be computed.

$$\begin{bmatrix} Y(0) \\ Y(1) \\ Y(2) \end{bmatrix} = \begin{bmatrix} Y(3) \\ Y(4) \\ Y(5) \end{bmatrix} = \begin{bmatrix} Y(6) \\ Y(7) \\ Y(8) \end{bmatrix} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & W_3 & W_3^2 \\ 1 & W_3^2 & W_3 \end{bmatrix} \begin{bmatrix} x(0) \\ x(3) \\ x(6) \end{bmatrix} \quad (17)$$

Only the last two entries $Y(1)$ and $Y(2)$ are needed from (17) to complete (12) using (14)

$$\begin{bmatrix} X(1) \\ X(2) \\ X(4) \\ X(5) \\ X(7) \\ X(8) \end{bmatrix} = \begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \\ \bar{X}(4) \\ \bar{X}(5) \\ \bar{X}(7) \\ \bar{X}(8) \end{bmatrix} + \begin{bmatrix} Y(1) \\ Y(2) \\ Y(1) \\ Y(2) \\ Y(1) \\ Y(2) \end{bmatrix}$$

This length 9 example thus requires a length 6 circular convolution and 2 length 3 DFT's, which can be computed using length 2 circular convolutions. If $N = 3^3$, we have a length $3^2 \cdot 2 = 18$ circular convolution and two length $3^2 = 9$ DFT's. These can be reduced to 2 length 6 convolutions and 4 length 3 DFT's, which are calculated with length 2 circular convolutions. If $N = p^r$ the length N transform is computed with 1 length $p^{r-1}(p-1)$ convolution, 2 length $p^{r-2}(p-1)$ convolutions, 4 length $p^{r-3}(p-1)$ convolutions, 8 length $p^{r-4}(p-1)$ convolutions $\dots$, terminating with 2^{r-1} length $p-1$ convolutions.

III. CONVOLUTION WITH THE MINIMUM NUMBER OF MULTIPLICATIONS

The prime and prime power DFT algorithms are means of converting the calculations required in the DFT into circular convolution. Then, special fast cyclic convolution techniques may be used to perform the calculations. An algorithm for computing a short length cyclic convolution in the minimum number of multiples for small values of N is based on recent work by Winograd [3].

Winograd's theorem on the minimum number of multiplications is explained in terms of polynomial multiplication. To cyclicly convolve the sequences $h_0, h_1, \dots, h_{N-1}$ and $x_0, x_1, \dots, x_{N-1}$ we need only find the N coefficients of the polynomial

$$Y(z) = H(z) \cdot X(z) \bmod (z^N - 1) \quad (18)$$

where

$$H(z) = \sum_{k=0}^{N-1} h_k z^k \quad X(z) = \sum_{k=0}^{N-1} x_k z^k \quad (19)$$

If $z^N - 1$ is written in terms of the K factors which are irreducible over the rationals

$$z^N - 1 = \prod_{i=1}^K Q_i(z) \quad (20)$$

with no common factors in the $Q_i(z)$ polynomials, Winograd's theorem states that the minimum number of multiplications required to compute the circular convolution of two length N sequences is $2N - K$. This theorem does not count multiplication by rational numbers.

In order to reduce the computation required for (18), $Y(z)$ is decomposed into K simpler parts using the polynomial version of the Chinese remainder theorem [4].

$$Y(z) = \left[\sum_{i=1}^K Y_i(z) S_i(z) \right] \bmod (z^N - 1) \quad (21)$$

$$Y_i(z) = H_i(z) X_i(z) \bmod (Q_i(z)), \quad i = 1, \dots, K \quad (22)$$

$$X_i(z) = X(z) \bmod (Q_i(z)), \quad i = 1, \dots, K$$

$$H_i(z) = H(z) \bmod (Q_i(z)), \quad i = 1, \dots, K. \quad (23)$$

The polynomials $S_i(z)$ $i = 1, \dots, K$ play the role of a Kronecker delta

$$S_i(z) \equiv 1 \bmod Q_i(z), \quad i = 1, \dots, K$$

$$S_i(z) \equiv 0 \bmod Q_j(z) \quad \forall j \neq i. \quad (24)$$

The $S_i(z)$ may be found by applying Euclid's algorithm to polynomials [4]. For example, to do cyclic convolution of the length 6 sequences $h_0, h_1, \dots, h_5$ and $x_0, x_1, \dots, x_5$, form

$$H(z) = h_0 + h_1 z + \dots + h_5 z^5$$

$$X(z) = x_0 + x_1 z + \dots + x_5 z^5.$$

First, the irreducible factors $Q_i(z)$ are found:

$$z^6 - 1 = (z+1)(z-1)(z^2+z+1)(z^2-z+1)$$

$$Q_1(z) = z+1, Q_2(z) = z-1, Q_3(z) = z^2+z+1,$$

$$Q_4(z) = z^2-z+1.$$

Next, the intermediary polynomials X_i , H_i , and Y_i are formed:¹

$$X_1(z) = X(z) \bmod (z+1) = x_0^1$$

$$= x_0 - x_1 + x_2 - x_3 + x_4 - x_5$$

$$X_2(z) = X(z) \bmod (z-1) = x_0^2$$

$$= x_0 + x_1 + x_2 + x_3 + x_4 + x_5$$

$$X_3(z) = X(z) \bmod (z^2+z+1) = x_0^3 + x_1^3 z$$

¹Superscripts are used to identify the i th polynomial.

$$= (x_0 - x_2 + x_3 - x_5) + (x_1 - x_2 + x_4 - x_5)z$$

$$X_4(z) = X(z) \bmod (z^2 - z + 1) = x_0^4 + x_1^4 z$$

$$= (x_0 - x_2 - x_3 + x_5) + (x_1 + x_2 - x_4 - x_5)z. \quad (25)$$

(The $H_i(z)$ polynomials are the same in form.)

$$Y_1(z) = H_1(z)X_1(z) \bmod (z + 1) = y_0^1 = h_0^1 x_0^1$$

$$Y_2(z) = H_2(z)X_2(z) \bmod (z - 1) = y_0^2 = h_0^2 x_0^2$$

$$Y_3(z) = H_3(z)X_3(z) \bmod (z^2 + z + 1) = y_0^3 + y_1^3 z$$

$$= (h_0^3 x_0^3 - h_1^3 x_1^3) + (h_1^3 x_0^3 + h_0^3 x_1^3 - h_1^3 x_1^3)z$$

$$Y_4(z) = H_4(z)X_4(z) \bmod (z^2 - z + 1) = y_0^4 + y_1^4 z$$

$$= (h_0^4 x_0^4 - h_1^4 x_1^4) + (h_1^4 x_0^4 + h_0^4 x_1^4 + h_1^4 x_1^4)z. \quad (26)$$

Now, the $Y_1(z)$ and $Y_2(z)$ polynomials are formed directly with one multiplication each. So, we have the intermediate variables

$$m_1 = h_0^1 x_0^1 \quad \text{and} \quad m_2 = h_0^2 x_0^2$$

giving

$$y_0^1 = m_1 \quad \text{and} \quad y_0^2 = m_2. \quad (27)$$

The polynomials $Y_3(z)$ and $Y_4(z)$ require only three multiplications each—similar to complex multiplication done in three real multiples. Thus, for $Y_3(z)$ we need

$$m_3 = (h_0^3 - h_1^3)(x_1^3 - x_0^3), \quad (28)$$

$$m_4 = h_0^3 x_0^3, \quad \text{and} \quad m_5 = h_1^3 x_1^3 \quad (29)$$

from which we obtain

$$y_1^3 = m_3 + m_4 \quad \text{and} \quad y_0^3 = m_4 - m_5.$$

For $Y_4(z)$ we need

$$m_6 = (h_0^4 + h_1^4)(x_0^4 + x_1^4), \quad m_7 = h_0^4 x_0^4, \quad \text{and} \quad m_8 = h_1^4 x_1^4 \quad (30)$$

which we use to form $y_1^4 = m_6 - m_7$ and $y_0^4 = m_7 - m_8$. At this point, we have the 4 component polynomials: $Y_1(z) = y_0^1$, $Y_2(z) = y_0^2$,

$$Y(z) = [Y_1(z)S_1(z) + Y_2(z)S_2(z) + Y_3(z)S_3(z) + Y_4(z)S_4(z)] \bmod (z^6 - 1). \quad (31)$$

The $S_i(z)$ polynomials satisfying (24) are

$$S_1(z) = -\frac{1}{6}(z^5 - z^4 + z^3 - z^2 + z - 1)$$

$$S_2(z) = \frac{1}{6}(z^5 + z^4 + z^3 + z^2 + z + 1)$$

$$S_3(z) = \frac{1}{6}(z^6 - z^5 - z^4 + 2z^3 - z^2 - z + 1)$$

$$S_4(z) = \frac{1}{6}(z^6 + z^5 - z^4 - 2z^3 - z^2 + z + 1). \quad (32)$$

In order to show the exact operations on the original $\{x_i\}$ and $\{h_i\}$, the vector of coefficients of $Y(z)$,

$$y = \begin{bmatrix} y_0 \\ y_1 \\ \vdots \\ y_{N-1} \end{bmatrix}$$

may be expressed in terms of

$$x = \begin{bmatrix} x_0 \\ x_1 \\ \vdots \\ x_{N-1} \end{bmatrix} \quad \text{and} \quad h = \begin{bmatrix} h_0 \\ h_1 \\ \vdots \\ h_{N-1} \end{bmatrix}$$

with two equations using $\otimes$ to indicate point by point multiplication of column vectors:

$$m = [B]h \otimes [A]x \quad (33)$$

$$y = [C]m. \quad (34)$$

A length N circular convolution requiring M multiplies can always be expressed this way with A and B $M \times N$ matrices and C an $N \times M$ matrix [4].

In order to put our example for $N = 6$ in this matrix form, we first identify the intermediate m parameters from (26), as shown in (27), (28), (29), and (30). The terms in (26) involving h and x are expanded according to (25) to obtain the matrices A and B .

$$m = \begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ 1 & -1 & 0 & 1 & -1 & 0 \\ 1 & 0 & -1 & 1 & 0 & -1 \\ 0 & 1 & -1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 & -1 & 0 \\ 1 & 0 & -1 & -1 & 0 & 1 \\ 0 & 1 & 1 & 0 & -1 & -1 \end{bmatrix} \begin{bmatrix} h_0 \\ h_1 \\ h_2 \\ h_3 \\ h_4 \\ h_5 \end{bmatrix} \otimes \begin{bmatrix} 1 & -1 & 1 & -1 & 1 & -1 \\ 1 & 1 & 1 & 1 & 1 & 1 \\ -1 & 1 & 0 & -1 & 1 & 0 \\ 1 & 0 & -1 & 1 & 0 & -1 \\ 0 & 1 & -1 & 0 & 1 & -1 \\ 1 & 1 & 0 & -1 & -1 & 0 \\ 1 & 0 & -1 & -1 & 0 & 1 \\ 0 & 1 & 1 & 0 & -1 & -1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \end{bmatrix}. \quad (35)$$

$$Y_3(z) = y_0^3 + y_1^3 z, \quad \text{and} \quad Y_4(z) = y_0^4 + y_1^4 z.$$

The final step is to express the polynomial $Y(z)$ in terms of the components $Y_i(z)$. We have

$$Y_1(z) = m_1$$

$$Y_2(z) = m_2$$

$$Y_3(z) = (m_4 - m_5) + (m_3 + m_4)z$$

$$Y_4(z) = (m_7 - m_8 + (m_6 - m_7)z. \quad (36)$$

Substitute (36) and (32) into (31), collect powers of z to obtain

$$y = C m$$

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \end{bmatrix} = \frac{1}{6} \begin{bmatrix} 1 & 1 & -1 & 1 & -2 & 1 & 1 & -2 \\ -1 & 1 & 2 & 1 & 1 & 2 & -1 & -1 \\ 1 & 1 & -1 & -2 & 1 & 1 & -2 & 1 \\ -1 & 1 & -1 & 1 & -2 & -1 & -1 & 2 \\ 1 & 1 & 2 & 1 & 1 & -2 & 1 & 1 \\ -1 & 1 & -1 & -2 & 1 & -1 & 2 & 1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \\ m_3 \\ m_4 \\ m_5 \\ m_6 \\ m_7 \\ m_8 \end{bmatrix} \quad (37)$$

We now have a length 6 cyclic convolution computed with 8 multiplications, which is the minimum number as given by Winograd's theorem:

$$2N - K = 2(6) - 4 = 8.$$

The multiplications by the rational numbers in A , B , and C are not counted.

The calculation of $Y_3(z)$ and $Y_4(z)$ may be done with a number of different algorithms. These will give values for m_3 through m_8 different from (28)-(30), and different evaluations of $Y_3(z)$ and $Y_4(z)$ in terms of the m 's. This freedom in the choice of calculating $Y_3(z)$ and $Y_4(z)$ may be used to try to minimize the number of adds for the convolution algorithm. No attempt has been made here to minimize the number of adds.

Convolution algorithms which achieve $2N - K$ multiplies and have simple A , B , and C matrices are only known for short lengths. For longer convolutions it is difficult to see how to keep the number of additions under control.

IV. APPLICATION TO THE DFT

The polynomial method of generating a circular convolution algorithm which requires the minimum number of multiplications has been used effectively for short lengths. Winograd [5] states that all known algorithms for computing circular convolution in the minimum number of multiplications require a large number of additions when $z^N - 1$ has large irreducible factors. Therefore, to be of practical interest, the DFT length will be kept small to take advantage of the change to a circular convolution. DFT algorithms for small values of N have been written with these methods for use in a prime factor FFT, to be described in Section V. These algorithms are different from those used for the nested DFT proposed by Winograd and described by Silverman [10] which is also described in Section V. Table I shows the number of operations required for short transforms intended for use in the prime factor FFT and for short transforms intended for use in the nested algorithm. Explicit formulas for short trans-

TABLE I
SHORT LENGTH DFT OPERATIONS COUNT

N	Prime Factor FFT			Nested Algorithm		
	Multiplies	Shifts	Adds	Multiplies	W^0 multiplies	Adds
2	0	0	2	0	2	2
3	1	1	6	2	1	6
4	0	0	8	0	4	8
5	4	2	17	5	1	17
7	8	0	36	8	1	36
8	2	0	26	2	6	26
9	8	2	49	10	2	49

forms to be used with a prime factor FFT are given for lengths 3, 5, 7, and 9 in the Appendix. These formulas come from combining the correction terms with special convolution algorithms described in Section III. Short transforms for use with the nesting algorithm are derived by modifying the transforms in the Appendix to reduce the number of W^0 multiplies as in (45)-(47).

To see how a DFT is implemented with a convolution algorithm, consider the following length 3 example:

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \end{bmatrix} = \begin{bmatrix} W^0 & W^0 & W^0 \\ W^0 & W^1 & W^2 \\ W^0 & W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \end{bmatrix}.$$

The convolution

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \end{bmatrix} = \begin{bmatrix} W^1 & W^2 \\ W^2 & W^1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(2) \end{bmatrix} \quad (38)$$

provides $\bar{X}(1)$, $\bar{X}(2)$, and

$$\begin{aligned} X(0) &= W^0 [x(0) + x(1) + x(2)] \\ X(1) &= W^0 x(0) + \bar{X}(1) \\ X(2) &= W^0 x(0) + \bar{X}(2). \end{aligned} \quad (39)$$

To get an explicit formula for the DFT, the convolution (38) is written in matrix form as in (33) and (34).

$$\begin{bmatrix} m_1 \\ m_2 \end{bmatrix} = \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} W^1 \\ W^2 \end{bmatrix} \otimes \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} x(1) \\ x(2) \end{bmatrix} \quad (40)$$

$$\begin{bmatrix} \bar{X}(1) \\ \bar{X}(2) \end{bmatrix} = \frac{1}{2} \begin{bmatrix} 1 & 1 \\ 1 & -1 \end{bmatrix} \begin{bmatrix} m_1 \\ m_2 \end{bmatrix}. \quad (41)$$

In applying (40) and (41) to the length 3 DFT we absorb the factor of $\frac{1}{2}$ into the B matrix. Then, combining this convolution with (39) gives the following length 3 DFT algorithm for use in the prime factor FFT:

$$\begin{aligned} a_1 &= x(1) + x(2), \\ a_2 &= x(1) - x(2), \\ a_3 &= x(0) + a_1; \end{aligned} \quad (42)$$

$$\begin{aligned} m_1 &= -\frac{1}{2} a_1, \\ m_2 &= -j \frac{\sqrt{3}}{2} a_2; \end{aligned} \quad (43)$$

$$\begin{aligned}
c_1 &= x(0) + m_1, \\
X(0) &= a_3, \\
X(1) &= c_1 + m_2, \\
X(2) &= c_1 - m_2.
\end{aligned} \tag{44}$$

Now, for the algorithm to be used in the nested DFT method, the multiplies by W^0 must be accounted for as explained in Section V. Therefore, we wish to minimize the multiplies by W^0 as well. This may be done by modifying the above length 3 DFT as shown below:

$$\begin{aligned}
a_1 &= x(1) + x(2), \\
a_2 &= x(1) - x(2), \\
a_3 &= x(0) + a_1;
\end{aligned} \tag{45}$$

$$\begin{aligned}
m_1 &= \left(-\frac{1}{2} - 1\right) a_1 = -\frac{3}{2} a_1, \\
m_2 &= -j \frac{\sqrt{3}}{2} a_2, \\
m_3 &= W^0 a_3 = 1 \cdot a_3;
\end{aligned} \tag{46}$$

$$\begin{aligned}
c_1 &= m_3 + m_1, \\
X(0) &= m_3, \\
X(1) &= c_1 + m_2, \\
X(2) &= c_1 - m_2.
\end{aligned} \tag{47}$$

The algorithm used for the prime factor FFT has 1 multiply, 1 shift (multiplication by $\frac{1}{2}$), and 6 adds, as shown in Table I. The algorithm used for the nested transform has 2 multiplies, 1 multiply by W^0 , and 6 additions. For complex data, the values in Table I are only doubled because the coefficients formed from the BW portion of the convolution are pure real or pure imaginary numbers. This occurs since the B matrix is such that the W 's occur as sums or differences of conjugate pairs.

These short length DFT algorithms may be written in a matrix form as

$$X = O D I x \tag{48}$$

where I is the $\mu \times N$ matrix representation of the input adds in (42) or (45), D is a $\mu \times \mu$ diagonal matrix of the multiply coefficients in (43) or (46), and O is the $N \times \mu$ matrix representation of the output adds in (44) or (47). For our length 3 example in (45)-(47),

$$\begin{bmatrix} X(0) \\ X(1) \\ X(2) \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 1 & -1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & -\frac{3}{2} & 0 \\ 0 & 0 & -j\frac{\sqrt{3}}{2} \end{bmatrix} \begin{bmatrix} 1 & 1 & 1 \\ 0 & 1 & 1 \\ 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} x(0) \\ x(1) \\ x(2) \end{bmatrix}.$$

The summation form of these matrices gives another way to represent these algorithms:

$$X(k) = \sum_{l=0}^{\mu-1} o_{kl} d_l \sum_{n=0}^{N-1} i_{ln} x(n). \tag{49}$$

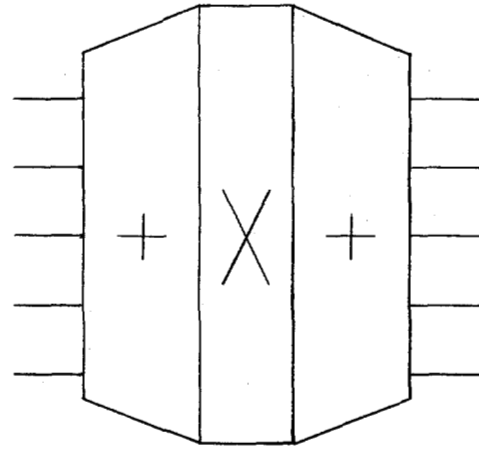


Fig. 1. DFT implemented with convolution showing expansion caused by more than 1 multiply per point.

In general, $\mu > N$ as shown by the expansion of the block labeled "X" in Fig. 1. The input and output additions are indicated by blocks labeled with a "+".

V. LONG TRANSFORMS FROM SHORT

The short transforms described above can be combined in several ways to provide a long transform of length N . The idea is to convert a one-dimensional length $N = M_1 M_2 \cdots M_l$ transform into an l -dimensional transform requiring computation of l shorter, length M_k transforms for $k = 1, \dots, l$. In this paper, we use a mapping from one to l dimensions which requires that the M_k factors be relatively prime [11]. Conventional FFT algorithms map one dimension to many dimensions. The Cooley-Tukey algorithm [1] allows common factors in N , while algorithms proposed by Good [6] and Thomas [7] use a mapping based on the Chinese remainder theorem [12] which requires relatively prime factors. We will use the Chinese remainder mapping which will be described for two factors M_1 and M_2 which are relatively prime.

In the DFT

$$X(k) = \sum_{n=0}^{N-1} x(n) W^{nk},$$

the index n of the input sequence $x(n)$ will be called the "input index," and the index k of the output sequence $X(k)$ will be called the "output index." The mapping from one-to-two dimensions maps the input index n into a pair of indices (n_1, n_2) .

$$\begin{aligned}
n_1 &= r_1 n \bmod M_1 & n_1 &= 0, 1, \dots, M_1 - 1 & r_1 &= M_2 \bmod M_1 \\
n_2 &= r_2 n \bmod M_2 & n_2 &= 0, 1, \dots, M_2 - 1 & r_2 &= M_1 \bmod M_2.
\end{aligned}$$

The output index is similarly mapped

$$\begin{aligned}
k_1 &= k \bmod M_1 & k_1 &= 0, 1, \dots, M_1 - 1 \\
k_2 &= k \bmod M_2 & k_2 &= 0, 1, \dots, M_2 - 1.
\end{aligned}$$

The inverse mapping from two dimensions to one dimension for the output index is

$$k = (s_1 k_1 + s_2 k_2) \bmod N \tag{50}$$

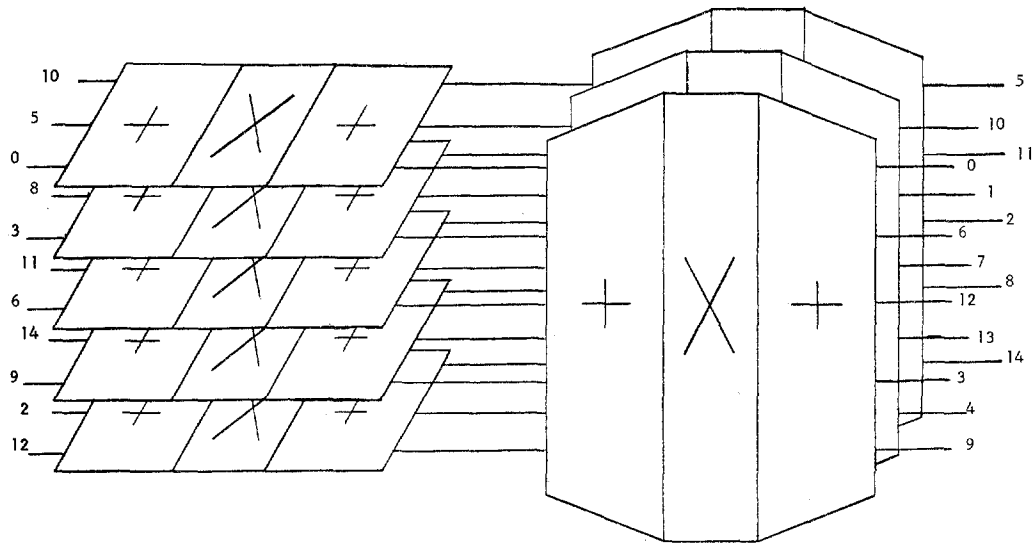


Fig. 2. Multidimensional prime factor FFT algorithm for length 15.

where

$$\begin{aligned} s_1 &\equiv 1 \pmod{M_1} \quad \text{and} \quad s_2 \equiv 0 \pmod{M_1} \\ s_1 &\equiv 0 \pmod{M_2} \quad \quad s_2 \equiv 1 \pmod{M_2}. \end{aligned}$$

While the same inverse mapping could be used for the input index, it is more convenient to use

$$n = (M_2 n_1 + M_1 n_2) \pmod{N}. \quad (51)$$

When these mappings are used, the DFT becomes

$$X(k_1, k_2) = \sum_{n_1=0}^{M_1-1} \sum_{n_2=0}^{M_2-1} x(n_1, n_2) W_{M_2}^{n_2 k_2} W_{M_1}^{n_1 k_1} \quad (52)$$

where

$$W_{M_1} = e^{-j(2\pi/M_1)}$$

and

$$W_{M_2} = e^{-j(2\pi/M_2)}.$$

The two-dimensional transform in (52) may be implemented by first calculating M_1 length M_2 DFT's,

$$y(n_1, k_2) = \sum_{n_2=0}^{M_2-1} x(n_1, n_2) W_{M_2}^{n_2 k_2}, \quad (53)$$

then calculating M_2 length M_1 DFT's

$$X(k_1, k_2) = \sum_{n_1=0}^{M_1-1} y(n_1, k_2) W_{M_1}^{n_1 k_1}. \quad (54)$$

The short transforms in (53) and (54) can be implemented using convolution methods as in (49). This procedure will be called the prime factor FFT algorithm and is illustrated in Fig. 2.

Fig. 2 shows a length 15 transform implemented by first calculating 5 length 3 transforms as in (50), then calculating 3 length 5 transforms as in (50). As in Fig. 1, blocks labeled "+" indicate addition, blocks labeled "X" indicate multiplication in the convolution DFT of (48). Fig. 2 is based on similar diagrams in Gold and Rader [13].

Winograd [5] has proposed another implementation of (52), which uses the special structure (49) of the short transforms to nest all multiplications inside of input and output additions. When the length M_1 short transform is written in terms of input additions $i^{(1)}$, output additions $o^{(1)}$, and multiplications $d^{(1)}$ and the length M_2 transform uses $i^{(2)}$, $o^{(2)}$, and $d^{(2)}$ as in (49), (53) becomes

$$y(n_1, k_2) = \sum_{l=0}^{\mu_2-1} o_{k_2 l}^{(2)} d_l^{(2)} \sum_{n_2=0}^{M_2-1} i_{ln_2}^{(2)} x(n_1, n_2). \quad (55)$$

Since $X(k_1, k_2)$ in (52) is a length M_1 transform of $y(n_1, k_2)$ which can also be implemented as in (49), (54) becomes

$$X(k_1, k_2) = \sum_{m=0}^{\mu_1-1} o_{k_1 m}^{(1)} d_m^{(1)} \sum_{n_1=0}^{M_1-1} i_{mn_1}^{(1)} Y(n_1, k_2). \quad (56)$$

Substituting (55) into (56), we get

$$\begin{aligned} X(k_1, k_2) = & \sum_{m=0}^{\mu_1-1} o_{k_1 m}^{(1)} d_m^{(1)} \sum_{n_1=0}^{M_1-1} i_{mn_1}^{(1)} \sum_{l=0}^{\mu_2-1} o_{k_2 l}^{(2)} d_l^{(2)} \\ & \cdot \sum_{n_2=0}^{M_2-1} i_{ln_2}^{(2)} x(n_1, n_2). \end{aligned} \quad (57)$$

The summations in (57) are an explicit representation of the operations indicated in Fig. 2. The order of summation may be changed as shown by McClellan and Rader [9] to nest all multiplies in the center giving

$$\begin{aligned} X(k_1, k_2) = & \sum_{l=0}^{\mu_2-1} o_{k_2 l}^{(2)} \sum_{m=0}^{\mu_1-1} o_{k_1 m}^{(1)} d_m^{(1)} d_l^{(2)} \sum_{n_1=0}^{M_1-1} i_{mn_1}^{(1)} \\ & \cdot \sum_{n_2=0}^{M_2-1} i_{ln_2}^{(2)} x(n_1, n_2). \end{aligned} \quad (58)$$

As shown in Fig. 3, (58) corresponds to first doing input adds on the rows of $x(n_1, n_2)$, then input adds on columns. Rows and columns are then multiplied as indicated by $d_m^{(1)}$

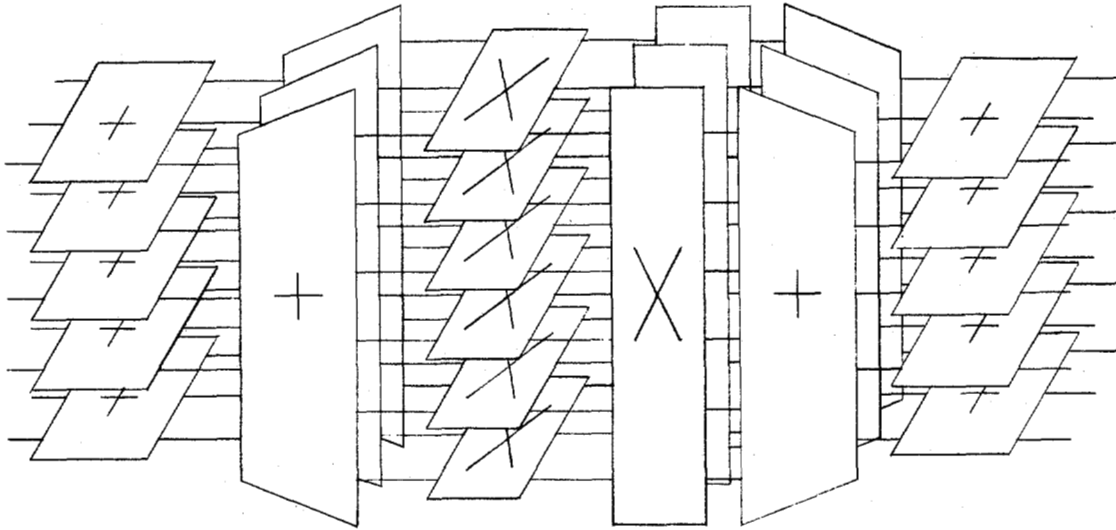


Fig. 3. Rearrangement of operations to nest multiplies inside of adds.

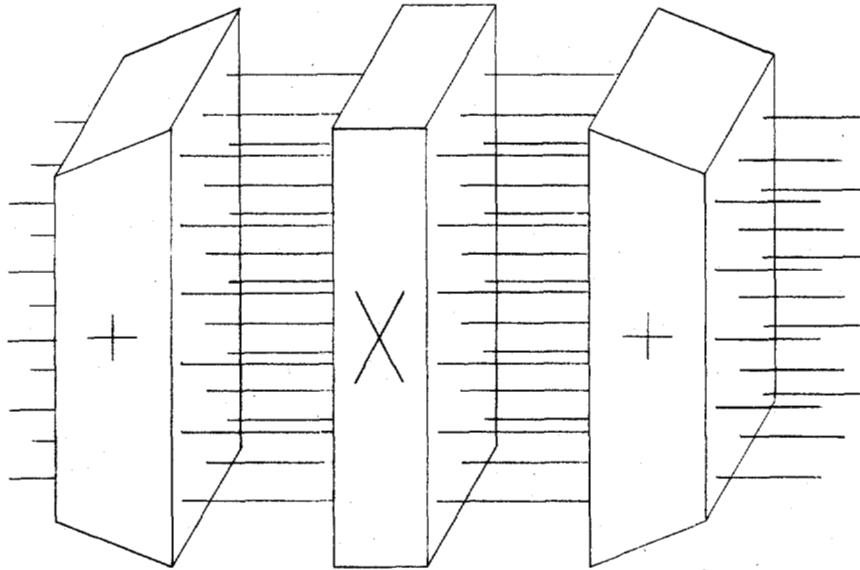


Fig. 4. Length 15 DFT implemented by the nested algorithm.

and $d_l^{(2)}$. Finally, output additions are performed on columns and rows. This algorithm, proposed by Winograd [5], will be called the nested algorithm in this paper because all multiplies are nested inside of additions as shown in Fig. 4.

The number of multiplications required by the nested algorithm is essentially the product of the total number of multiplications for each factor in the DFT, when implemented as in (58). However, some savings may be made for the case where the DFT being computed is at a point by point multiplication involving a product of W^0 coefficients. This corresponds to both d 's in (58) being unity. The number of multiplications saved is the product of the number of W^0 multiplies in each short transform. Thus, the equation for the number of multiplications for 2 factors is $\# \text{ multiplies} = \mu_1 \mu_2 - \kappa_1 \kappa_2$, where

- κ_i # of multiplies by W^0 for length M_i DFT,
- μ_i total # of multiplies for length M_i DFT,
- α_i # of additions.

For three factors we have $\# \text{ multiplies} = \mu_1 \mu_2 \mu_3 - \kappa_1 \kappa_2 \kappa_3$, and the pattern continues for more factors.

An equation for the number of additions is based on structures of the nested algorithms in (58) and on Fig. 3. The horizontal addition planes in Fig. 3 correspond to length M_2 transforms and there are M_1 of them (the indices n_1 and k_1 take on M_1 values). This contributes $M_1 \alpha_2$ additions. The vertical addition planes in Fig. 3 correspond to length M_1 transforms and there are μ_2 of them (the index l takes on μ_2 values). This contributes $\mu_2 \alpha_1$ additions. Thus, for two factors we have

$$\# \text{ additions} = M_1 \alpha_2 + \mu_2 \alpha_1.$$

For three factors we have

$$\# \text{ additions} = M_1 M_2 \alpha_3 + \mu_3 [M_1 \alpha_2 + \mu_2 \alpha_1].$$

In the same manner, we have with four factors

TABLE II
COMPARISON OF DFT ALGORITHMS

N	Nested Algorithm			Prime Factor FFT		Radix 2 FFT	
	Factors	Multiplies	Adds	Multiplies	Adds	Multiplies	Adds
30	$5 \cdot 3 \cdot 2$	68	384	68	384		
32	2^5					102	830
126	$9 \cdot 7 \cdot 2$	424	3312	512	2920		
128	2^7					774	5662
252	$9 \cdot 7 \cdot 4$	848	7128	1024	6344		
256	2^8					1926	13 726
315	$9 \cdot 5 \cdot 7$	1292	11 286	1784	8812		
360	$9 \cdot 5 \cdot 8$	1152	9492	1396	8708		
504	$9 \cdot 7 \cdot 8$	1704	15 516	2300	13 948		
512	2^9					4614	32 286
840	$5 \cdot 7 \cdot 8 \cdot 3$	2592	24 804	4244	23 172		
1024	2^{10}					10 758	74 270
1260	$9 \cdot 5 \cdot 7 \cdot 4$	5168	50 184	7136	40 288		
2048	2^{11}					24 582	167 966
2520	$9 \cdot 5 \cdot 7 \cdot 8$	10 344	106 667	15 532	86 876		

$$\begin{aligned} \# \text{ additions} = & M_1 M_2 M_3 \alpha_4 + \mu_4 [M_1 M_2 \alpha_3 \\ & + \mu_3 [M_1 \alpha_2 + \mu_2 \alpha_1]]. \end{aligned} \quad (59)$$

For the number of multiplications, the ordering of the factors is unimportant. However, the number of additions required depends on the ordering of the factors. For the nested transform additions given in Table II, the best ordering was used and is indicated by the order of factors. For complex data, the number of real multiplications and real additions is twice the numbers given by the above equations (multiplications of the complex data are with pure real or pure imaginary coefficients).

To compare the operations count of the nested algorithm with the operations required by the Cooley-Tukey and the prime factor FFT algorithm, we need to look at the latter two algorithms and determine the number of operations they require. In order to make the comparison more realistic, the radix 2 Cooley-Tukey algorithm will perform complex multiplies in three real multiplications and will not count multiplies by W^0 . Multiplications by $\pm j$ will not be counted. We have for $N = 2^M$ ($\log N = M$) [14]:

$$\begin{aligned} \# \text{ multiplies} &= 3 \left[\frac{N}{2} \log N - \frac{3N}{2} + 2 \right] \\ \# \text{ adds} &= 2N \log N + 5 [\# \text{ multiplies}]. \end{aligned}$$

For the prime factor FFT algorithm we use the special short length transforms intended for this algorithm. See the first section of Table I. With $N = M_1 M_2 M_3$, $M_2 M_3$ length M_1 transforms are computed, $M_1 M_3$ length M_2 DFT's and $M_1 M_2$ length M_3 DFT's. We have

$$\# \text{ multiplies} = 2 [M_2 M_3 (\mu_1) + M_1 M_3 (\mu_2) + M_1 M_2 (\mu_3)] \quad (60)$$

$$\# \text{ adds} = 2 [M_2 M_3 (\alpha_1) + M_1 M_3 (\alpha_2) + M_1 M_2 (\alpha_3)] \quad (61)$$

for complex data. Using these equations, the three algorithms are compared in Table II for several values of N .

VI. COMPARISON OF DFT ALGORITHMS—DISCUSSION OF TABLE II

The comparison in Table II shows that the prime factor FFT algorithm requires from 0–64 percent more multiplies than the nested transform. However the nested transform requires from 0–28 percent more additions than the prime factor FFT algorithm. In fact, if additions “cost” at least one-half as much as multiplications, then the multiply-add cost for the prime factor FFT algorithm is smaller for all lengths shown in Table II, except for lengths 30 and 840.

To develop an understanding for how these two transforms are related for various choices of factors, we will derive expressions for the number of operations required per output point.

From (60), the number of multiplies per output point for the prime factor FFT algorithm is simply the sum of the number of multiplies per point for each factor.

$$\# \text{ multiplies per point} = \sum \left(\frac{\mu_i}{M_i} \right). \quad (62)$$

Similarly, for the prime factor FFT algorithm,

$$\# \text{ adds per point} = \sum \left(\frac{\alpha_i}{M_i} \right). \quad (63)$$

For the nested algorithm, the number of multiplies per point is approximately the product of the number of multiplies per point for each factor.

$$\# \text{ multiplies per point} = \prod_i \left(\frac{\mu_i}{M_i} \right). \quad (64)$$

From (59), the number of additions per point for the nested transform is

$$\begin{aligned} \# \text{ adds per point} = & \frac{\alpha_1}{M_1} + \left(\frac{\mu_1}{M_1} \right) \frac{\alpha_2}{M_2} + \left(\frac{\mu_1}{M_1} \right) \left(\frac{\mu_2}{M_2} \right) \frac{\alpha_3}{M_3} \\ & + \dots \end{aligned} \quad (65)$$

For the factors used in this study, the number of operations per point is shown in Table III.

With k factors and an average number of multiplies per point, μ , the nested algorithm requires μ^k multiplies per point. The prime factor FFT algorithm requires $k\mu$ multiplies per point. When $\mu^k > k\mu$ or when $\mu > {}^{k-1}\sqrt{k}$, the prime factor FFT algorithm requires fewer multiplies than the nested algorithm.

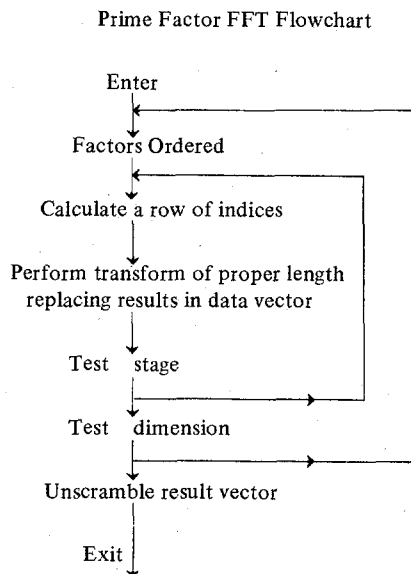
Since ${}^{k-1}\sqrt{k}$ becomes smaller with increasing k (more factors) and μ_i increases for the extra factors which must be large, the prime factor FFT algorithm will have fewer multiplies per point than the nested algorithm when more factors are used.

With k factors and average number of additions per point, α , the prime factor FFT algorithm requires $k\alpha$ additions per point. The nested algorithm requires $\alpha + \mu\alpha + \dots + \mu^{k-1}\alpha = (\mu^k - 1)/(\mu - 1) \cdot \alpha$ additions per point for $\mu > 1$. In the special case $N = 2^r \cdot 3 \cdot p$ where p is a prime other than 2 or 3 and $r = 1, 2$, or 3, both algorithms require the same number additions since $\mu = 1$ for the factors 2^r and 3. With other factors the prime factor FFT algorithm will have fewer additions. As shown above the difference in the number of additions will also increase rapidly when more factors are used.

A good strategy would be to use nesting for a few factors until μ began to grow, then combine, using the prime factor FFT algorithm, with another composite length short transform which was done with nesting.

VII. PROGRAMS FOR COMPARING TRANSFORM METHODS

The prime factor FFT algorithm was used to program a mixed radix DFT in which the short length DFT's are calculated using the fast convolution method previously described. A flowchart of the program is shown below.



For $N = M_1 \cdot M_2 \cdot M_3$, there are three dimensions in the transform. First, $M_2 \cdot M_3$ length M_1 transforms are calculated. The indices of the data required for each length M_1 transform are calculated using the input mapping equation for the prime factor FFT algorithm (51). Then, the data corresponding to these indices is transformed by a length M_1 DFT using the fast

TABLE III
PRIME FACTOR FFT NESTED ALGORITHM

N	μ/N	α/N	μ/N	α/N
2	0	1.0	1.0	1.0
3	0.33	2.0	1.0	2.0
4	0	2.0	1.0	2.0
5	0.8	3.4	1.2	3.4
7	1.14	5.14	1.28	5.14
8	0.25	3.25	1.0	3.25
9	0.89	5.44	1.33	5.44

TABLE IV
TIME TEST ON IBM 370/155

N	Times in Seconds			Percentage of Time Savings
	Prime Factor FFT	Singleton's FFT	Radix 2 FFT	
32			0.013	
60	0.017	0.025		47 percent
64			0.027	
128			0.059	
210	0.080	0.119		49 percent
256			0.129	
315	0.111	0.179		61 percent
504	0.168	0.288		71 percent
512			0.280	
840	0.344	0.509		48 percent
1024			0.609	
1260	0.540	0.809		50 percent
2048			1.323	
2520	1.115	1.714		54 percent

convolution technique. The results of this transform are replaced in the data vector. Then, if more length M_1 transforms are required, the new indices are calculated. If all the length M_1 DFT's are finished, the program moves on to calculating the $M_1 \cdot M_3$ length M_2 DFT's. Finally, when the DFT's along all the dimensions have been calculated, the vector of the results must be unscrambled. This is done using the output mapping for the Good algorithms given by (50). The length N transform is then complete.

This prime factor FFT algorithm for calculating the DFT was programmed in Fortran and in 8080 microprocessor assembly language. The Fortran prime factor FFT was compared in speed to a mixed radix FFT program written by Singleton [15], which uses a Cooley-Tukey mapping. The FFT subroutine of Singleton is 50 percent longer than the prime factor FFT subroutine. However, the prime factor FFT uses storage of two vectors of length N , while the Singleton FFT subroutine requires one length N vector. The results of the time tests for several transform lengths are given in Table IV. These tests were run on an IBM 370 computer for which the ratio of multiply to add time was 3. The power of 2 algorithm was taken from Rabiner and Gold [14]. It may be 15 percent slower than an algorithm which stores all powers of W .

The timing for the subroutines was accomplished using an interval timer on the IBM 370. The percent saving in time given is the percent by which the Singleton FFT subroutine is slower than the prime factor FFT subroutine.

The timing results of Table IV are for calculating the frequency response of a length 32 finite impulse response digital bandpass filter and are taken from a single run.

The times for the prime factor FFT may be calculated from the Table II values for the total number of operations by the following formula:

$$\begin{aligned} \text{time (in ms)} = & (NF(0.052) + 0.028)N \\ & + 0.0096 (\# \text{ multiplies}) + 0.0045 (\# \text{ adds}). \end{aligned}$$

The input indexing for the program took $52 \mu\text{s}$ per point for each dimension of the transform (NF equals the number of factors). The output indexing took $28 \mu\text{s}$ per point. The code generated by the Fortran add and multiply statements took 4.5 and $9.6 \mu\text{s}$, respectively, to run. In the program, the shifts in the short DFT algorithms for the prime factor FFT were done as multiplies.

Next, the 8080 microprocessor assembly language version of the prime factor FFT subroutine was compared in speed with a radix 2 FFT subroutine. The radix 2 FFT was also written in assembly language, used three real multiplies for each complex multiply, and did not multiply by W^0 . In addition, the FFT used precalculated values of W_N^i which were stored in a table. The FFT program was much shorter than the prime factor FFT program. The ratio of multiply to add times on the 8080 was approximately 30. A length 252 Good DFT requires 3.20 seconds. A length 256 radix 2 FFT requires 5.42 seconds. So, the radix 2 FFT subroutine is 70 percent slower than the prime factor FFT subroutine. The savings occur in both the multiplications and additions. The fewer multiplications required make up approximately 80 percent of the savings, the remainder of which is addition savings.

VIII. CONCLUSIONS

The conversion of a DFT into a circular convolution leads to new methods for computation of the DFT.

For short transforms these algorithms require few multiplications and additions as shown in Table I and as shown in the explicit formulas given in the Appendix.

Long transforms are built up from these short transforms in several ways which are compared in Table II. The prime factor FFT algorithm was chosen as the most attractive approach for several reasons. The prime factor FFT algorithm has about the same combined total of multiply-adds as the nested algorithm. However, it is easier to write a general prime factor FFT program. The prime factor FFT can be calculated using less memory than is required for the nested algorithm. It requires less data storage and probably less program memory. Since the prime factor FFT algorithm is done in small pieces it might run faster on machines with small high speed memory blocks. Special hardware for parallel computation will probably be simpler for the prime factor FFT algorithm.

A general prime factor FFT program was written for an IBM 370 in Fortran and for an 8080 microprocessor in assembly language, and the running time for this new algorithm was compared with a conventional FFT. In the 370 comparison the new algorithm was compared with the mixed radix algorithm of Singleton [15], since the prime factor FFT algorithm is a mixed radix algorithm. A reduction of approximately 50 percent was observed (see Table IV). Much larger savings may be expected if special hardware is constructed for the short convolution-based algorithms.

Many open questions remain. How should one combine nesting and prime factor FFT techniques to obtain long transforms from short ones? Should the multidimensional expansion always be done at the transform level, or should the convolutions contained within transforms also be implemented in multidimensional expansions? How can one improve on the indexing schemes required for these new transforms? It is likely that continuing development of longer and more efficient convolution algorithms will make implementations of the DFT using convolution even more attractive.

APPENDIX

3 Point DFT Algorithm

$$\begin{aligned} a_1 &= x(1) + x(2) & m_1 &= \frac{1}{2} a_1 & c_1 &= x(0) - m_1 \\ a_2 &= x(1) - x(2) & m_2 &= 0.86603 a_2 \\ a_3 &= x(0) + a_1 & & & X(0) &= a_3 \end{aligned}$$

1 Multiply, 6 Adds, 1 Shift.

$$\begin{aligned} X(1) &= c_1 - jm_2 \\ X(2) &= c_1 + jm_2 \end{aligned}$$

5 Point DFT Algorithm

$$\begin{aligned} a_1 &= x(1) + x(4) & m_1 &= 0.95106 a_5 & c_1 &= x(0) - m_5 \\ a_2 &= x(1) - x(4) & m_2 &= 1.53884 a_2 & c_2 &= c_1 + m_4 \\ a_3 &= x(2) + x(3) & m_3 &= 0.36327 a_4 & c_3 &= c_1 - m_4 \\ a_4 &= x(2) - x(3) & m_4 &= 0.55902 a_6 & c_4 &= m_1 - m_3 \\ a_5 &= a_2 + a_4 & m_5 &= \frac{1}{4} a_7 & c_5 &= m_2 - m_1 \\ a_6 &= a_1 - a_3 & & & & \end{aligned}$$

$$a_7 = a_1 + a_3$$

$$a_8 = x(0) + a_7$$

4 Multiplies, 17 Adds, 2 Shifts.

7 Point DFT Algorithm

$$a_1 = x(1) + x(6)$$

$$a_2 = x(1) - x(6)$$

$$a_3 = x(2) + x(5)$$

$$a_4 = x(2) - x(5)$$

$$a_5 = x(3) + x(4)$$

$$a_6 = x(3) - x(4)$$

$$a_7 = a_1 + a_3 + a_5$$

$$a_8 = a_1 - a_5$$

$$a_9 = -a_3 + a_5$$

$$a_{10} = -a_1 + a_3$$

$$a_{11} = a_2 + a_4 - a_6$$

$$a_{12} = a_2 + a_6$$

$$a_{13} = -a_4 - a_6$$

$$a_{14} = -a_2 + a_4$$

$$a_{15} = x(0) + a_7$$

8 Multiplies, 36 Adds.

9 Point DFT Algorithm

$$a_1 = x(1) + x(8)$$

$$a_2 = x(1) - x(8)$$

$$a_3 = x(2) + x(7)$$

$$a_4 = x(2) - x(7)$$

$$a_5 = x(4) + x(5)$$

$$a_6 = x(4) - x(5)$$

$$a_7 = x(3) + x(6)$$

$$a_8 = x(3) - x(6)$$

$$a_9 = -a_1 + a_5$$

$$a_{10} = a_1 - a_3$$

$$a_{11} = -a_3 + a_5$$

$$a_{12} = a_2 - a_6$$

$$a_{13} = a_2 + a_4$$

$$a_{14} = -a_4 - a_6$$

$$a_{15} = a_1 + a_3 + a_5$$

$$a_{16} = a_2 - a_4 + a_6$$

$$m_1 = 0.16667 a_7$$

$$m_2 = 0.79016 a_8$$

$$m_3 = 0.05585 a_9$$

$$m_4 = 0.73430 a_{10}$$

$$m_5 = 0.44096 a_{11}$$

$$m_6 = 0.34087 a_{12}$$

$$m_7 = 0.53397 a_{13}$$

$$m_8 = 0.87484 a_{14}$$

$$X(0) = a_8$$

$$X(1) = c_2 - jc_4$$

$$X(2) = c_3 - jc_5$$

$$X(3) = c_3 + jc_5$$

$$X(4) = c_2 + jc_4$$

$$c_1 = x(0) - m_1$$

$$c_2 = c_1 + m_2 + m_3$$

$$c_3 = c_1 - m_2 - m_4$$

$$c_4 = c_1 - m_3 + m_4$$

$$c_5 = m_5 + m_6 - m_7$$

$$c_6 = m_5 - m_6 - m_8$$

$$c_7 = -m_5 - m_7 - m_8$$

$$X(0) = a_{15}$$

$$X(1) = c_2 - jc_5$$

$$X(2) = c_3 - jc_6$$

$$X(3) = c_4 - jc_7$$

$$X(4) = c_4 + jc_7$$

$$X(5) = c_3 + jc_6$$

$$X(6) = c_2 + jc_5$$

$$c_1 = x(0) - m_7$$

$$c_2 = m_2 - m_3$$

$$c_3 = m_1 + m_3$$

$$c_4 = m_1 + m_2$$

$$c_5 = c_1 + c_2 - c_3$$

$$c_6 = c_1 + c_3 + c_4$$

$$c_7 = c_1 - c_2 - c_4$$

$$c_8 = m_4 - m_6$$

$$c_9 = m_5 - m_6$$

$$c_{10} = m_4 - m_5$$

$$c_{11} = c_8 + c_9 + m_8$$

$$c_{12} = c_8 + c_{10} - m_8$$

$$c_{13} = -c_9 + c_{10} + m_8$$

$$c_{14} = x(0) + a_7 - m_9$$

$$X(0) = a_{17}$$

$$X(1) = c_5 - jc_{11}$$

$$a_{17} = x(0) + a_{15} + a_7$$

8 Multiplies, 49 Adds, 2 Shifts.

$$X(2) = c_6 - jc_{12}$$

$$X(3) = c_{14} - jm_{10}$$

$$X(4) = c_7 - jc_{13}$$

$$X(5) = c_7 + jc_{13}$$

$$X(6) = c_{14} + jm_{10}$$

$$X(7) = c_6 + jc_{12}$$

$$X(8) = c_5 + jc_{11}$$

ACKNOWLEDGMENT

The authors would like to acknowledge the suggestions and helpful comments of C. M. Rader and J. H. McClellan.

REFERENCES

- [1] J. W. Cooley and J. W. Tukey, "An algorithm for the machine calculation of complex Fourier series," *Math. Comput.*, vol. 19, pp. 297-301, Apr. 1965.
- [2] C. M. Rader, "Discrete Fourier transforms when the number of data samples is prime," *Proc. IEEE*, vol. 56, pp. 1107-1108, June 1968.
- [3] S. Winograd, "Some bilinear forms whose multiplicative complexity depends on the field of constants," IBM T. J. Watson Res. Ctr., Yorktown Heights, NY, IBM Res. Rep., RC 5669, Oct. 10, 1975.
- [4] R. Agarwal and J. W. Cooley, "New algorithms for digital convolution," presented at the IEEE Arden House Workshop, Feb. 1976; also to appear in this TRANSACTIONS.
- [5] S. Winograd, "On computing the discrete Fourier transform," *Proc. Nat. Acad. Sci. USA*, vol. 73, no. 4, pp. 1005-1006, Apr. 1976.
- [6] I. J. Good, "The interaction algorithm and practical Fourier analysis," *J. Royal Statist. Soc., ser. B*, vol. 20, pp. 361-372, 1958; "Addendum," vol. 22 (MR 21 1674; MR 23 A4231), pp. 372-375, 1960.
- [7] L. H. Thomas, "Using a computer to solve problems in physics," in *Applications of Digital Computers*. Boston, MA: Ginn and Co., 1963.
- [8] C. D. Mostow, J. H. Sampson, and J. Meyer, *Fundamental Structures of Algebra*. New York: McGraw-Hill, 1963.
- [9] J. McClellan and C. M. Rader, "There is something much faster than the fast Fourier transform," Seminar Notes, Oct. 21, 1976.
- [10] H. F. Silverman, "An introduction to programming the Winograd Fourier transform algorithm (WFTA)," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-25, pp. 152-165, Apr. 1977.
- [11] C. S. Burrus, "Index mappings for multidimensional formulation of the DFT and convolution," *IEEE Trans. Acoust., Speech, Signal Processing*, vol. ASSP-25, pp. 239-242, June 1977.
- [12] D. E. Knuth, "Seminumerical algorithms," in *The Art of Computer Programming*, vol. 2. Reading, MA: Addison-Wesley, 1971.
- [13] B. Gold and C. M. Rader, *Digital Processing of Signals*. New York: McGraw-Hill, 1969, p. 194.
- [14] L. R. Rabiner and B. Gold, *Theory and Application of Digital Signal Processing*. Englewood Cliffs, NJ: Prentice-Hall, 1975.
- [15] R. C. Singleton, "An algorithm for computing the mixed radix fast Fourier transform," *IEEE Trans. Audio Electroacoust.*, vol. AU-17, pp. 93-103, June 1969.